

Data Driven Approach for Handling Class Imbalance Problem in Credit Card Fraud Detection

*A Thesis Submitted in Partial Fulfilment
of the Requirements for the Degree of*

**Master of Technology
in Artificial Intelligence and Machine Learning**

by

Aditya Sharma

Roll No.: 24MCSA02

Under the supervision of

Dr. Ranjeet Kumar Ranjan

Department of Computer Science and Engineering



**Department of Computer Science and Engineering
PDPM Indian Institute of Information Technology,
Design and Manufacturing, Jabalpur**

June 2026

CERTIFICATE

This is to certify that the thesis titled “*Data Driven Approach for Handling Class Imbalance Problem in Credit Card Fraud Detection*” submitted by **Aditya Sharma** (Roll No.: 24MCSA02) to the Department of Computer Science and Engineering, PDPM Indian Institute of Information Technology, Design and Manufacturing, Jabalpur, for the award of the degree of **Master of Technology**, is a record of bona fide research work carried out by him under my supervision and guidance.

To the best of my knowledge, the work reported in this thesis has not been submitted to any other university or institute for the award of any degree or diploma.

Dr. Ranjeet Kumar Ranjan

Department of Computer Science and Engineering

PDPM IITDM Jabalpur

Date: _____

DECLARATION

I hereby declare that the thesis titled ‘*Data Driven Approach for Handling Class Imbalance Problem in Credit Card Fraud Detection*’ submitted by me to the Department of Computer Science and Engineering, PDPM Indian Institute of Information Technology, Design and Manufacturing, Jabalpur, for the award of the degree of **Master of Technology**, is a record of original research work carried out by me under the supervision of **Dr. Ranjeet Kumar Ranjan**.

The work presented in this thesis has not been submitted, either in part or in full, to any other university or institute for the award of any degree or diploma. All sources of information used in this thesis have been duly acknowledged.

Aditya Sharma

Roll No.: 24MCSA02

M.Tech (Artificial Intelligence and Machine Learning)

PDPM IIITDM Jabalpur

Date: _____

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, **Dr. Ranjeet Kumar Ranjan**, for his constant guidance, valuable suggestions, and continuous encouragement throughout the course of this work. His insights helped me at every stage of the research, from picking the problem to analysing the results. His patience during difficult phases of the work meant a lot to me.

I am also thankful to the Head of the Department and to all the faculty members of the Department of Computer Science and Engineering, PDPM IIITDM Jabalpur, for providing the necessary facilities and a healthy academic environment for carrying out this research. I extend my sincere thanks to the technical staff of the department for their support with the computing resources used in this work.

I would like to thank my friends and classmates at PDPM IIITDM Jabalpur who supported me throughout my M.Tech journey. The countless discussions, debugging sessions, and shared cups of tea kept me going.

Above all, I am deeply grateful to my parents and family members for their love, support, and constant motivation. Without their belief in me, this work would not have been possible.

Aditya Sharma

ABSTRACT

Credit card fraud causes billions of dollars in losses every year. One of the fundamental challenges in building automated fraud detection systems is that fraud cases are extremely rare compared to legitimate transactions. This severe class imbalance makes standard machine learning models biased toward predicting every transaction as legitimate, which results in high overall accuracy but very poor fraud detection – exactly the opposite of what is needed in practice.

This thesis addresses the class imbalance problem in credit card fraud detection by using Generative Adversarial Networks to generate realistic synthetic fraud samples. Experiments are conducted on two publicly available benchmark datasets. The first is the **ULB Credit Card Fraud dataset**, containing real-world transactions made by European cardholders, which is widely used as a standard benchmark for this problem. The second is the **IEEE-CIS Fraud Detection dataset**, a larger and more complex dataset released by IEEE Computational Intelligence Society and Vesta Corporation, containing richer transaction and identity features that represent a more realistic fraud detection scenario.

The experimental study follows a structured pipeline. We begin with exploratory data analysis and preprocessing for both datasets. We then evaluate five standard classifiers (Logistic Regression, K-Nearest Neighbors, Decision Tree, Random Forest, and XGBoost) on the raw imbalanced data to establish a baseline. We next apply three traditional oversampling methods – Random Oversampling (ROS), SMOTE, and ADASYN – and measure their effect on classifier performance.

We then evaluate two GAN-based oversampling approaches – Vanilla GAN and CTGAN – at multiple augmentation ratios.

The core contribution of this thesis is the **Boundary-Aware Dual-Critic Generative Adversarial Network (BD-WGAN)**. A key limitation of existing GAN-based methods is that they generate synthetic samples covering the overall fraud distribution but miss the hard fraud cases near the decision boundary – fraud transactions that closely resemble legitimate ones. BD-WGAN addresses this by introducing a second critic network (the boundary critic) trained specifically to recognise these boundary-region fraud samples. The generator is optimised jointly by two critics: a Wasserstein distribution critic for overall realism, and a boundary critic for boundary coverage. A cluster-conditioning mechanism further accounts for the fact that fraud patterns are not homogeneous but fall into distinct sub-groups.

Experimental results on both datasets show that BD-WGAN achieves better recall and lower false negative count compared to all baseline and GAN-based methods. The method also outperforms several recently published approaches on the same bench-

marks.

Keywords: credit card fraud detection, class imbalance, generative adversarial network, WGAN-GP, boundary-aware learning, synthetic data augmentation, ULB dataset, IEEE-CIS dataset.

Contents

1	Introduction	2
1.1	Background	2
1.2	The Class Imbalance Problem	3
1.3	Generative Adversarial Networks	4
1.4	Problem Statement	4
1.5	Contributions of This Thesis	5
1.6	Thesis Organisation	6
2	Literature Review	7
2.1	Machine Learning for Fraud Detection	7
2.2	Handling Class Imbalance	8
2.2.1	Resampling Methods	8
2.2.2	Algorithm-Level Methods	9
2.2.3	Hybrid and Ensemble Methods	9
2.3	GAN-Based Oversampling for Fraud Detection	10
2.3.1	Early GAN Applications	10
2.3.2	Conditional and Tabular GANs	10
2.3.3	Hard Fraud and Boundary-Aware Approaches	10
2.4	Summary of Research Gaps	11
3	Background and Theoretical Concepts	13
3.1	Machine Learning Classifiers	13
3.1.1	Logistic Regression	13
3.1.2	K-Nearest Neighbours	13
3.1.3	Decision Tree	14
3.1.4	Random Forest	14
3.1.5	XGBoost	14
3.2	Evaluation Metrics	15
3.2.1	The Problem with Accuracy	15
3.2.2	Recall	15
3.2.3	Precision	16
3.2.4	F1 Score	16

3.2.5	PR-AUC and ROC-AUC	16
3.3	Oversampling: Theoretical Background	16
3.3.1	Random Over-Sampling	16
3.3.2	SMOTE	17
3.3.3	ADASYN	17
3.4	Generative Adversarial Networks	17
3.4.1	Standard GAN	17
3.4.2	WGAN-GP	18
3.4.3	CTGAN	18
3.5	Supporting Tools	19
3.5.1	PCA	19
3.5.2	K-Means Clustering	19
3.5.3	Kolmogorov-Smirnov Test	19
3.5.4	t-SNE	19
4	Dataset, Preprocessing and Baseline Methods	21
4.1	Dataset Description	21
4.1.1	ULB Credit Card Fraud Dataset	21
4.1.2	IEEE-CIS Fraud Detection Dataset	22
4.2	Exploratory Data Analysis	22
4.2.1	Data Quality	22
4.2.2	Discriminative Features	22
4.2.3	Internal Structure of Fraud (t-SNE)	23
4.3	Preprocessing Pipeline	24
4.3.1	ULB Preprocessing	24
4.3.2	IEEE-CIS Feature Engineering and Preprocessing	25
4.4	Baseline Classifiers	26
4.5	Traditional Oversampling Methods	27
4.6	GAN-Based Oversampling Baselines	29
4.6.1	Vanilla GAN	29
4.6.2	CTGAN	30
4.6.3	Quality Check	30
4.6.4	Results	31
	Bibliography	32

List of Figures

4.1	Features ranked by KS statistic between fraud and legitimate classes (higher = stronger separation)	23
4.2	t-SNE projection: blue points are legitimate transactions, red are fraud. Fraud spreads across multiple separated regions rather than a single cluster.	24
4.3	Confusion matrices for the five baseline classifiers on the ULB test set	26
4.4	ROC and Precision-Recall curves for the five baseline classifiers . . .	27
4.5	Confusion matrices for all 15 resampler $\times$ classifier combinations . . .	28
4.6	Performance heatmaps (Recall, F1, PR-AUC) across sampler-classifier combinations	29
4.7	KS statistics per feature for each GAN (lower = generated distribution closer to real)	30
4.8	t-SNE projections comparing real fraud (blue) and synthetic fraud from each GAN (orange)	30

List of Tables

3.1	Confusion matrix cells for binary fraud detection	15
4.1	Class distribution in both datasets used in this thesis	22
4.2	Training/test split after ULB preprocessing	25
4.3	Baseline results on the ULB test set (56,749 transactions, 98 fraud). No resampling applied. Bold marks the lowest FN.	26
4.4	Full traditional resampling results on the ULB test set. Bold marks the lowest-FN result per sampler at Precision ≥ 0.70 . LR under ROS/ADASYN achieves FN=8 but at precision below 0.06 – over 1,000 false positives – which is not usable.	28
4.5	Best results per GAN across all 50 ratio $\times$ classifier configurations (Precision ≥ 0.70 , lowest FN).	31

LIST OF ABBREVIATIONS

GAN	Generative Adversarial Network
WGAN	Wasserstein Generative Adversarial Network
WGAN-GP	Wasserstein GAN with Gradient Penalty
BD-WGAN	Boundary-aware Dual-critic WGAN
CTGAN	Conditional Tabular Generative Adversarial Network
ROS	Random Over-Sampling
SMOTE	Synthetic Minority Over-sampling Technique
ADASYN	Adaptive Synthetic Sampling
LR	Logistic Regression
KNN	K-Nearest Neighbors
DT	Decision Tree
RF	Random Forest
XGB	XGBoost
PCA	Principal Component Analysis
KS	Kolmogorov-Smirnov (test)
t-SNE	t-distributed Stochastic Neighbor Embedding
ROC-AUC	Receiver Operating Characteristic – Area Under Curve
PR-AUC	Precision-Recall – Area Under Curve
FN	False Negative
FP	False Positive
TP	True Positive
TN	True Negative
MoG	Mixture of Gaussians
BCE	Binary Cross-Entropy
GP	Gradient Penalty
ULB	Universit�e Libre de Bruxelles

CHAPTER 1

Introduction

1.1 Background

In the last decade, credit card transactions have become one of the most widely used methods for financial and online transactions all over the world. Millions of financial transactions take place every day through online shopping, bill payments, and ATM withdrawals. As digital payments have grown, the problem of credit card fraud has also increased alongside them.

Card fraud is not a new problem, but it has changed a lot over the years. Earlier, most fraud happened by physically stealing a card and using it before the owner noticed. Now, the physical card barely matters. A fraudster needs only the card number, expiry date, and the three-digit security code to start making online purchases. These details get stolen through company data breaches, fake login pages that look like real bank websites, or bought cheaply from criminal online markets. This kind of fraud – called Card-Not-Present (CNP) fraud – now makes up about 79% of total card fraud losses in the European Union [9].

There are other forms too. Skimming devices placed on ATMs copy the magnetic strip data from cards swiped through them, allowing fraudsters to clone the card later. Account takeover exploits stolen passwords, often obtained through phishing or SIM-swapping attacks that redirect a user's phone number. Social engineering attacks involve fraudsters calling victims and pretending to be bank staff – they convince people to read out their OTP or PIN by creating a false sense of urgency.

The money lost to these attacks is not trivial. A 2022 industry report estimated global card fraud losses at around \$33.83 billion, with projections crossing \$38.5 billion by 2027 [25]. These losses fall on banks that must reimburse victims, on merchants who bear chargeback costs, and on cardholders who lose time and money dealing with the aftermath.

Banks want to stop fraudulent transactions before they complete. The challenge is that they must do this automatically, in real time, across millions of daily transactions, without blocking too many legitimate purchases. Machine learning has become the main tool for this, but it carries a practical problem that is surprisingly hard to deal with.

1.2 The Class Imbalance Problem

In any real fraud dataset, fraudulent transactions are a tiny fraction of the total. In the ULB dataset used in this thesis, only 492 out of 284,807 transactions are fraud. For every fraud case, there are about 577 legitimate ones.

This ratio causes a familiar problem. A machine learning model that simply labels every transaction as legitimate will be correct 99.8% of the time – but it catches no fraud at all. Standard training algorithms minimise overall error, and since nearly every sample is legitimate, the easiest way to minimise error is to predict legitimate for almost everything [13]. The model learns the wrong lesson from the data.

Missing fraud is expensive. The cardholder loses money, the bank pays the charge-back, and the fraudster gets away. But blocking legitimate transactions is also costly in a different way – customers stop trusting the bank’s card, move to a competitor, or simply get frustrated by declined payments at the checkout. A good fraud detection system has to walk a narrow line: catch as many real frauds as possible without flagging too many innocent transactions.

Several directions have been tried to fix this:

1. **Resampling the training data.** The basic idea is to change the training set so both classes are better represented. Random Over-Sampling (ROS) simply copies existing fraud samples until the counts balance. SMOTE [6] takes a step further by generating new synthetic samples placed on the line between two real fraud samples. ADASYN [14] improves on SMOTE by putting more synthetic samples near the fraud cases that are hardest to separate from legitimate ones. Undersampling cuts majority-class samples instead, but throws away information in the process.
2. **Adjusting the learning algorithm.** Here the training data stays as it is, but the model is told that misclassifying a fraud case costs more than misclassifying a legitimate one. Cost-sensitive learning [17] and class-weighted loss functions implement this idea. It helps, but only to a point.
3. **Generating synthetic data using deep generative models.** Instead of copying or interpolating, a generative model tries to actually learn what the fraud distribution looks like and then draw new samples from it. This adds genuine new examples rather than repetitions or line-segment approximations.

This thesis takes the third direction.

1.3 Generative Adversarial Networks

Generative Adversarial Networks (GANs) were introduced by Goodfellow et al. in 2014 [11]. The design involves two neural networks placed in competition with each other.

The Generator takes a random noise vector as input and tries to produce a synthetic sample that resembles real data. The Discriminator takes any sample – real or synthetic – and tries to judge which is which. Both are trained at the same time. The Generator wants to produce outputs that fool the Discriminator; the Discriminator wants to get better at detecting the Generator’s outputs. As training continues, the Generator is forced to produce more realistic samples or the Discriminator will keep catching it. In the ideal outcome, the Generator produces samples that are indistinguishable from real data.

For fraud detection, the plan is to train a GAN using only the fraud samples in the training set. The Generator learns what a fraud transaction looks like – the typical ranges of the PCA features, how the transaction amount behaves, what patterns the V-features show. Then it generates additional fraud samples, which are mixed into the training set before a downstream classifier is trained. The classifier gets to see more examples of fraud and should learn to detect it better.

The practical difficulties are real. Tabular data has no spatial structure, features can be skewed or multi-modal, and the GAN cannot rely on local patterns the way image GANs can. The original GAN training is also unstable – the Generator can collapse into producing only a few types of outputs (mode collapse) or the gradients can vanish entirely when the Discriminator becomes too confident.

Two improved variants are used in this work. WGAN-GP [12] replaces the original loss function with the Wasserstein distance and adds a gradient penalty term to the critic’s loss. This stabilises training considerably and is now the most widely used approach for practical GAN work on tabular data. CTGAN [27] handles multi-modal feature distributions in tabular data using variational Gaussian mixture normalisation. Both appear as comparison methods in the experiments.

1.4 Problem Statement

Looking at the published results on the ULB credit card fraud dataset, recall values across different studies range from 0.76 to about 0.95. Even the papers reporting the higher end of that range still miss between 5 and 25 fraud cases out of every 100. Each of those misses represents a real transaction where money was stolen and not caught.

Thinking about why these misses happen, many of them cluster around what might

be called hard fraud cases – fraud transactions that, when mapped into the feature space, look very similar to legitimate transactions. These sit near the decision boundary between the two classes. Standard GANs generate synthetic samples roughly in proportion to where the real fraud samples are. Since hard boundary-region fraud is sparse in the training data to begin with, the GAN produces few samples from that region, and the classifier is poorly trained on exactly the cases it will struggle with the most.

The core question this work tries to answer:

Can a GAN be designed to deliberately generate more samples from the hard, boundary-region fraud cases – and does doing this lead to fewer missed frauds, without increasing false alarms unacceptably?

1.5 Contributions of This Thesis

This thesis contributes the following:

1. Experimental evaluation on two benchmark datasets – the ULB Credit Card Fraud dataset and the IEEE-CIS Fraud Detection dataset – including dataset-specific preprocessing and exploratory analysis.
2. Baseline results for five standard classifiers (Logistic Regression, KNN, Decision Tree, Random Forest, XGBoost) on raw imbalanced data, establishing what is achievable without any intervention.
3. Results for three classical oversampling methods – ROS, SMOTE, ADASYN – combined with the same five classifiers.
4. Results for two GAN-based augmentation methods – Vanilla GAN and CTGAN – at five different augmentation ratios (75 configurations total).
5. The proposed **BD-WGAN** (Boundary-aware Dual-Critic Wasserstein GAN): a GAN with a second critic that is specifically trained on hard boundary-region fraud samples and guides the generator to produce more of them.
6. A cluster-conditioning mechanism that divides fraud samples into sub-groups by K-Means clustering, and conditions the generator on the cluster label to preserve the internal diversity of the fraud class.
7. A progressive improvement study across three versions of BD-WGAN (v1 to v3), each addressing identified weaknesses in synthetic data quality.

8. Comparison with recently published methods showing BD-WGAN improves recall over all baseline and traditional methods while maintaining precision above the usable threshold.

1.6 Thesis Organisation

The rest of this thesis is arranged as follows:

- **Chapter 2 – Literature Review** covers related work in fraud detection and GAN-based augmentation, and identifies the specific gaps that motivate the proposed method.
- **Chapter 3 – Background and Theoretical Concepts** explains all five classifiers, the evaluation metrics, the three oversampling methods, and the three GAN variants used in experiments, along with supporting tools like PCA, K-Means, the KS test, and t-SNE.
- **Chapter 4 – Dataset, Preprocessing and Baseline Methods** introduces both datasets, describes all preprocessing decisions, and presents results for baselines, traditional resampling, and GAN-based augmentation.
- **Chapter 5 – Proposed Work: BD-WGAN** presents the full architecture, training procedure, and design reasoning for the proposed method.
- **Chapter 6 – Results and Conclusion** compares all methods, benchmarks against recent literature, states conclusions and limitations, and outlines directions for future work.

CHAPTER 2

Literature Review

This chapter looks at what has already been done in credit card fraud detection and where the current work falls short. Three threads of research are covered: machine learning classifiers applied to fraud, methods for handling class imbalance, and GAN-based synthetic data generation. Reading through these studies makes the specific gap that BD-WGAN tries to fill much clearer.

2.1 Machine Learning for Fraud Detection

Early work on automated fraud detection applied whatever classifiers were available at the time. Logistic regression and naive Bayes were popular starting points in the late 1990s and early 2000s, mainly because they trained quickly, were easy to explain to risk managers, and ran fast enough for real-time scoring on the hardware of that era. Neither assumption – linearity in logistic regression, feature independence in naive Bayes – holds well in real transaction data, but the models still worked well enough to be useful as a first filter.

Tree-based methods displaced them as the standard approach for tabular data. Breiman’s Random Forest [5] showed that averaging over many trees trained on different data subsets reduces variance dramatically, and the random feature selection at each split gives it robustness against correlated features. Bhattacharyya et al. [4] compared logistic regression, neural networks, and Random Forest on a real-world fraud dataset and found the ensemble approach consistently better, particularly as the imbalance ratio increased. Sahin and Duman [23] reported similar findings using decision trees and shallow neural networks across several card fraud datasets.

XGBoost [7] pushed gradient boosting forward by adding L_1 and L_2 regularisation, a smarter handling of missing values, and several implementation tricks that made it faster than earlier boosting implementations. It won or placed in most Kaggle competitions on tabular data between 2015 and 2020 and has become the default benchmark that fraud detection papers compare against. Dal Pozzolo et al. [8] worked directly with the ULB dataset and showed through careful analysis that even XGBoost suffers significantly when class imbalance is extreme – the model works well at high transaction volumes but degrades badly on the rare fraud cases.

Deep learning has been tried for fraud detection as well. Roy et al. [22] built a multi-layer neural network for the task and reported small improvements over classi-

cal methods. Pumsirirat and Yan [21] used an autoencoder trained only on legitimate transactions; fraud was detected as samples with unusually high reconstruction error. This anomaly detection approach sidesteps the imbalance problem entirely but tends to struggle when fraud transactions happen to look similar to legitimate ones in the autoencoder’s learned representation. Across the published work, deep models have not shown a consistent advantage over Random Forest or XGBoost on the standard fraud benchmark, possibly because the dataset is too small for deep models to generalise well from such few fraud examples.

The common thread across this body of work: better classifiers help, but they do not fix the imbalance. Every study that reports strong numbers on the ULB dataset either handles the imbalance separately or focuses on metrics that hide the problem (accuracy). This pushed the research community toward imbalance-specific methods.

2.2 Handling Class Imbalance

2.2.1 Resampling Methods

The most straightforward fix is to change the training data rather than the model. Random Over-Sampling copies existing minority samples at random until both classes are even. The problem is obvious in hindsight: if the model just sees the same 492 fraud transactions over and over, it does not learn anything new about fraud. It may in fact learn to recognise those specific transactions rather than general fraud patterns. This is sometimes called overfitting to the training set’s particular fraud instances.

Random Under-Sampling cuts majority samples down to match the minority count. Removing over 280,000 legitimate transactions to keep pace with 492 fraud cases throws out an enormous amount of information about what normal behaviour looks like. Not surprisingly, this tends to hurt precision badly.

SMOTE, proposed by Chawla et al. [6], was the paper that changed how people approached this problem. Instead of copying existing samples, SMOTE creates new synthetic ones by picking a minority sample, finding its nearest minority-class neighbours, and placing new points along the line segments connecting them. The new samples are not exact duplicates, so the model is exposed to slightly different fraud patterns on each training run. SMOTE became the most widely cited oversampling method and spawned dozens of variants.

ADASYN, introduced by He et al. [14], noticed that SMOTE treats all minority samples equally when deciding where to generate new ones. ADASYN weights minority samples by how many of their neighbours belong to the majority class – samples surrounded by majority neighbours are harder to learn and get more synthetic sam-

ples near them. The intuition is that the classifier needs more examples precisely where the two classes are mixed together, not where fraud is already easy to separate. Alfaizi and Fati [2] used ADASYN with Random Forest on the ULB dataset and achieved a recall of 0.95, which is the highest reported in that line of work, though precision fell to 0.78.

The limitation of all these methods is the same: they generate samples on straight lines between existing training points. If the fraud distribution has multiple peaks or non-convex structure – which t-SNE visualisation of the ULB data suggests it does – then samples drawn from linear interpolations between those peaks can land in regions that correspond to no real fraud pattern.

2.2.2 Algorithm-Level Methods

Some researchers prefer not to touch the data at all. Cost-sensitive learning [17] tells the model to penalise misclassifying a minority sample more heavily than a majority one. In practice this means setting the `class_weight` parameter in scikit-learn classifiers or the `scale_pos_weight` parameter in XGBoost. It is computationally free and easy to implement. The drawback is that it requires choosing the penalty weight carefully – too high and the model becomes too aggressive, flagging everything as fraud; too low and it ignores the rare class anyway.

Anomaly detection approaches – Isolation Forest, One-Class SVM, local outlier factor – train only on majority-class data and flag minority samples as outliers. These can work well when fraud is genuinely rare and anomalous, but they fail when fraudsters deliberately mimic normal transaction patterns to avoid detection.

2.2.3 Hybrid and Ensemble Methods

Some studies combine both directions. SMOTE-ENN applies SMOTE oversampling and then removes ambiguous samples near the boundary using Edited Nearest Neighbours. SMOTE-Tomek does the same with Tomek links. The idea is that SMOTE may place some synthetic samples in the wrong region, and the cleaning step removes the most problematic ones. Results on fraud data are modest improvements over plain SMOTE.

Ensemble approaches train multiple models on different balanced subsets. Liu et al. [17] proposed EasyEnsemble and BalancedBagging, both of which build classifiers on different undersampled versions of the majority class and combine their votes. A more recent study trained bagging and stacking ensembles with CTGAN-generated data, reporting recall 0.76 and precision 0.94 on the ULB dataset [1]. The high precision at that recall level is encouraging but the recall itself is still below what the best oversampling methods achieve.

2.3 GAN-Based Oversampling for Fraud Detection

2.3.1 Early GAN Applications

Fiore et al. [10] were among the first to apply a GAN specifically to credit card fraud. They trained a standard vanilla GAN on the fraud class and used the generated samples to augment training data for a logistic regression classifier. Recall improved compared to no augmentation, which showed the concept was viable. The vanilla GAN training was unstable and required careful tuning, but the result encouraged follow-up work.

Ngwenduna and Mbuva [19] replaced the vanilla GAN with a WGAN-GP variant, motivated by the known stability advantages of the Wasserstein objective. Their experiments found improved recall when training XGBoost on the WGAN-GP-augmented data, though precision dropped and F1 did not show clear improvement across all settings. The stability of training did improve, but getting the right augmentation ratio – how many synthetic samples to add – still required tuning.

2.3.2 Conditional and Tabular GANs

Xu et al.’s CTGAN [27] was a turning point for GAN-based work on tabular data. By using variational Gaussian mixture normalisation for each continuous feature and conditional sampling for categorical columns, CTGAN could handle the kinds of distributions that appear in real transaction datasets – features with multiple peaks, heavy tails, and zero-inflated counts. Patil [20] applied CTGAN to ULB with Random Forest and XGBoost, reporting F1 around 0.84. This was better than vanilla GAN results but still behind the best traditional oversampling results.

Liu et al. [18] proposed CTCN, which combines CTGAN for synthetic generation with a Temporal Convolutional Network classifier to exploit sequential patterns in transaction data. They reported F1 scores around 0.87 on ULB. The improvement over CTGAN-only approaches came largely from the stronger classifier rather than from better synthetic samples.

Some recent papers report F1 above 0.99 on ULB using combinations of GANs, autoencoders, and denoising methods. These results are difficult to reproduce based on the published methodology and are well above what any independent study has achieved. They are treated with caution here.

2.3.3 Hard Fraud and Boundary-Aware Approaches

A smaller thread of work has focused specifically on the fraud cases that are most difficult to classify. Lin et al.’s focal loss [16] was developed for object detection but

applies directly to imbalanced classification: it down-weights the loss contribution from easy, well-classified samples so training focuses on the hard ones. Hard example mining [24] in object detection does something similar by explicitly selecting hard examples for each training iteration.

ADASYN implicitly applies this idea at the data level by weighting minority samples by their proximity to the majority class boundary. However, it is still constrained to linear interpolation. What has not been done in the fraud literature is building a *generator* that is explicitly guided to produce boundary-region samples. Most GAN-based methods generate samples proportional to the overall minority density – they produce many samples from common fraud patterns and few from the rare boundary cases.

This is the gap that the proposed method fills.

2.4 Summary of Research Gaps

Reading through the existing work, four problems stand out that have not been properly addressed:

Gap 1 – GANs do not target hard fraud. When trained on fraud samples, a GAN generates new ones roughly in proportion to where the existing fraud is dense. Hard boundary-region fraud – transactions that look nearly legitimate – is sparse in the training data, so the GAN produces few such samples. The downstream classifier ends up under-trained on exactly the cases it needs to handle.

Gap 2 – Interpolation methods cannot capture complex fraud structure. SMOTE and ADASYN place new samples on straight lines between existing ones. t-SNE visualisation shows that fraud does not lie on a convex set – it occupies multiple scattered regions. Synthetic samples generated on lines between points from different fraud sub-groups may not correspond to any real fraud pattern.

Gap 3 – Most work evaluates on only one dataset. Nearly all GAN-based fraud papers test on the ULB dataset alone. Whether the improvements transfer to other fraud datasets with different imbalance ratios, feature types, and fraud patterns remains largely untested.

Gap 4 – Synthetic sample quality is rarely evaluated separately. The standard evaluation compares classifier performance on original versus augmented data. If the synthetic samples are unrealistic, the classifier may learn spurious patterns from them. Very few papers separately check the statistical quality of generated samples before training a classifier.

The proposed BD-WGAN directly targets Gaps 1 and 2. The second critic explicitly guides the generator toward boundary-region fraud. The cluster-conditioning mechanism preserves the multi-modal structure of the fraud class. Experiments on

two datasets partly address Gap 3, and a dedicated synthetic quality evaluation using KS tests, MMD, and discriminator accuracy addresses Gap 4.

CHAPTER 3

Background and Theoretical Concepts

Before getting into the experiments, it helps to be clear about the tools used and why each one was chosen. This chapter covers three things: the five classifiers used in all experiments, the metrics used to measure their performance, and the generative models at the centre of this work. Some supporting tools – PCA, K-Means, the KS test, t-SNE – are explained at the end since they come up in the proposed method.

3.1 Machine Learning Classifiers

3.1.1 Logistic Regression

Logistic regression has been around since the 1950s and remains one of the most commonly used classification models in practice. It estimates the probability that an input belongs to the positive class using a sigmoid function on a linear combination of features:

$$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} \quad (3.1)$$

where $\mathbf{w}$ is the learned weight vector and b is the bias. The weights are found by minimising binary cross-entropy on the training set.

The model’s strength is its simplicity and speed. Given well-scaled features, it trains in seconds and its coefficients tell you directly which features push predictions toward fraud. The weakness is equally straightforward: the decision boundary is a flat hyperplane in feature space. Fraud patterns in real data are not linearly separable, so logistic regression leaves a lot of predictive power on the table. In the experiments here, it serves as the lowest-capability baseline.

3.1.2 K-Nearest Neighbours

KNN does not build any model during training. When a new transaction needs to be classified, it searches the entire training set for the k most similar transactions by Euclidean distance and assigns the class that the majority of those neighbours belong to. With $k = 5$ as used here, the prediction is the majority vote of the five closest training examples.

The advantage is that KNN can model any decision boundary shape without any assumptions about the data distribution. The disadvantage is cost: every prediction requires computing distances to all training samples, which becomes slow when the training set is large. KNN is also sensitive to feature scale – a feature with large numeric values will dominate the distance calculation unless the data is standardised first. `StandardScaler` normalisation is always applied before using KNN in this work.

3.1.3 Decision Tree

A decision tree partitions the feature space by a sequence of threshold comparisons. At each node, the algorithm searches over all features and all possible split values for the one that reduces impurity (measured by Gini impurity or information gain) the most. The process continues recursively until each leaf is pure or a stopping criterion is met.

Trees are the most interpretable classifier: the whole model can be drawn as a diagram and a prediction can be traced step by step. The problem is that an unconstrained tree memorises the training data. This matters for fraud detection because the handful of fraud samples in training can produce unreliable splits that do not generalise to new fraud. Pruning and depth limits help but do not fully solve it.

3.1.4 Random Forest

Random Forest [5] builds many decision trees and combines their predictions by majority vote. Two sources of randomness are introduced: each tree is trained on a bootstrap sample of the data (sampling with replacement), and at each split only a random subset of features is considered rather than all of them.

These two tricks together reduce variance dramatically. Individual trees overfit; their average does not. The random feature selection also de-correlates the trees – if a few strong features dominate, standard bagging would produce similar trees that all make the same errors. Random Forest avoids this. On most tabular fraud datasets, Random Forest is the strongest performer among classical methods, and it is used as the primary baseline in this work.

3.1.5 XGBoost

XGBoost [7] builds trees sequentially. Each new tree is fitted to the residuals (more precisely, the gradients of the loss) of all the trees already in the ensemble. This is gradient boosting. XGBoost adds L_1 and L_2 regularisation terms to the objective to prevent individual trees from growing too complex, and uses a histogram-based

split-finding algorithm that makes it much faster than earlier gradient boosting implementations.

On tabular data, XGBoost is very hard to beat with default settings. It handles missing values natively, deals reasonably well with skewed features, and the regularisation limits overfitting even with hundreds of trees. It is used here with default settings and no class weighting in the baseline experiments, deliberately, to show what happens when imbalance is ignored.

3.2 Evaluation Metrics

3.2.1 The Problem with Accuracy

A classifier that labels every transaction as legitimate on the ULB dataset would be right 99.83% of the time. This is clearly wrong for the intended purpose. Accuracy hides everything that matters when classes are this imbalanced. All metrics used here are computed from the confusion matrix, which breaks the predictions down by actual and predicted class.

Table 3.1: Confusion matrix cells for binary fraud detection

	Predicted Legitimate	Predicted Fraud
Actual Legitimate	True Negative (TN)	False Positive (FP)
Actual Fraud	False Negative (FN)	True Positive (TP)

In fraud detection, a False Negative is worse than a False Positive. A missed fraud means money is lost; a false alarm means a legitimate transaction is briefly blocked. The cost ratio is not the same, and the metrics reflect this.

3.2.2 Recall

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3.2)$$

Recall is the fraction of real fraud cases that were caught. It is the primary metric throughout this thesis. When recall is 0.90, ten fraud cases out of every hundred slipped through. The raw FN count (missed frauds) is reported alongside recall since it is more directly interpretable: “missed 13 of 98 fraud cases” is clearer than “recall 0.867.”

3.2.3 Precision

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3.3)$$

Precision is the fraction of flagged transactions that are actually fraud. Low precision means many legitimate transactions are being blocked, which angers customers and wastes investigation time. A minimum precision of 0.70 is used when comparing configurations in this work – below that, the false alarm rate makes a configuration impractical regardless of how good the recall is.

3.2.4 F1 Score

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3.4)$$

F1 is the harmonic mean of precision and recall. It penalises extreme values in either direction – a high recall with terrible precision will give a low F1, same as the reverse. F1 is widely reported in fraud detection papers and is used here for comparisons, though it is a secondary metric since it weights precision and recall equally, whereas catching fraud is the priority.

3.2.5 PR-AUC and ROC-AUC

Both are summary metrics computed by varying the decision threshold from 0 to 1 and tracing out a curve.

ROC-AUC plots True Positive Rate against False Positive Rate. It is well-known and easy to interpret, but it can be misleading when the negative class is enormous – even large numbers of false positives can produce a small False Positive Rate when there are hundreds of thousands of true negatives.

PR-AUC plots Precision against Recall. Since it directly involves the positive class without the diluting effect of the huge negative class, it gives a more honest picture of performance under imbalance. PR-AUC is the secondary summary metric in this thesis, alongside the F1 score.

3.3 Oversampling: Theoretical Background

3.3.1 Random Over-Sampling

ROS randomly selects minority class samples and duplicates them until the desired ratio is reached. In the ULB dataset with 394 training fraud samples, reaching a 1:1 ratio requires duplicating each fraud sample roughly 575 times. The model then trains

on the same 394 original fraud transactions repeatedly. There is no new information; the only effect is that the gradient contributions from fraud and legitimate samples are weighted more evenly.

3.3.2 SMOTE

SMOTE [6] generates new minority samples by linear interpolation. For each minority sample x_i , it selects one of its k nearest minority neighbours x_{nn} and creates a new point:

$$x_{new} = x_i + \lambda \cdot (x_{nn} - x_i), \quad \lambda \sim \text{Uniform}(0, 1) \quad (3.5)$$

The new sample lies somewhere on the line segment between x_i and x_{nn} . This adds genuine diversity – new samples are not copies of existing ones. But the diversity is constrained to the convex hull of the minority class: no sample can be generated outside the region spanned by existing fraud points.

3.3.3 ADASYN

ADASYN [14] keeps the same synthesis formula as SMOTE but changes which samples get generated. For each minority sample, it computes the proportion of its k nearest neighbours that belong to the majority class. Samples surrounded by mostly majority neighbours are near the decision boundary and harder to classify. ADASYN assigns these samples a higher weight, so more synthetic samples are generated near them. The generation formula is identical to SMOTE; only the sampling distribution over minority examples changes.

3.4 Generative Adversarial Networks

3.4.1 Standard GAN

Goodfellow et al. [11] proposed the GAN framework as a way to train a generative model without having to specify an explicit density function. Two networks are trained together:

- The **Generator** $G(z; \theta_G)$ maps a noise vector $z \sim \mathcal{N}(0, I)$ to a synthetic sample $\tilde{x} = G(z)$.
- The **Discriminator** $D(x; \theta_D)$ maps any sample to a probability that it came from the real data rather than the generator.

Training alternates between updating D to better distinguish real from synthetic and updating G to fool D :

$$\min_G \max_D \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \quad (3.6)$$

At equilibrium, the theory says G perfectly reproduces p_{data} and D cannot do better than guessing at 0.5 for everything. In practice, training rarely reaches this point cleanly. The gradients for G can vanish when D becomes too confident, and the generator can collapse into producing only a small range of outputs regardless of the noise input – a failure called mode collapse.

3.4.2 WGAN-GP

Arjovsky et al. [3] replaced the Jensen-Shannon divergence in the original GAN with the Wasserstein-1 (Earth Mover’s) distance. The new objective uses a *critic* instead of a discriminator – the output is unbounded rather than clamped to $[0, 1]$:

$$\underbrace{\mathbb{E}_z[C(G(z))]}_{\text{fake score}} - \underbrace{\mathbb{E}_x[C(x)]}_{\text{real score}} + \lambda \mathbb{E}_{\hat{x}} [(\|\nabla_{\hat{x}} C(\hat{x})\|_2 - 1)^2] \quad (3.7)$$

The last term is the gradient penalty (from Gulrajani et al. [12]). It enforces the Lipschitz constraint on the critic by penalising gradients whose norm deviates from 1, where $\hat{x}$ is sampled uniformly on the straight line between a real and a generated sample. The penalty replaces the weight clipping used in the original WGAN, giving more stable gradients throughout training. WGAN-GP is the base architecture for the proposed BD-WGAN.

3.4.3 CTGAN

CTGAN [27] was built specifically because standard GANs perform poorly on tabular data. Two problems motivated the design.

Continuous features in tabular data often have multi-modal distributions. A fraud transaction amount might cluster near small values (everyday purchases), have a gap, and then have another cluster at medium values. A standard GAN normalises the whole distribution at once, which blurs these modes together. CTGAN fits a Variational Gaussian Mixture (VGM) model to each continuous feature and uses a one-hot mode indicator plus a rescaled value within the mode to represent each sample. The generator learns each mode separately.

Categorical features can be severely imbalanced. In a transaction dataset, 95% of purchases might be one product category and 5% another. A naive generator would almost never produce samples from the rare category. CTGAN uses a conditional

training procedure where the generator is conditioned on a randomly chosen category for each batch, ensuring all categories are represented in training. For this work, CTGAN is used as a comparison GAN baseline.

3.5 Supporting Tools

3.5.1 PCA

Principal Component Analysis reduces a high-dimensional dataset to a smaller number of orthogonal dimensions (principal components) that capture the most variance. The first component captures more variance than the second, the second more than the third, and so on. In this work, PCA to 10 components is applied to the fraud training samples before K-Means clustering. The 10-dimensional representation captures most of the variance in the 29-feature fraud data while making the clustering more stable.

3.5.2 K-Means Clustering

K-Means divides a dataset into k clusters by alternating between two steps: assigning each point to its nearest cluster centre and re-computing each centre as the mean of its assigned points. It stops when assignments do not change. The number of clusters k is selected here by maximising the silhouette score, which measures how similar each point is to its own cluster compared to the nearest other cluster. K-Means is applied to the PCA-reduced fraud samples to find distinct fraud sub-groups that the generator is conditioned on.

3.5.3 Kolmogorov-Smirnov Test

The KS test checks whether two one-dimensional samples come from the same distribution. The test statistic is the maximum absolute difference between the empirical cumulative distribution functions of the two samples. A small KS statistic means the distributions are similar. A large one means they are different. In this work, the KS test is applied to each feature separately to measure how well the synthetic fraud samples match the real fraud distribution – lower mean KS across all 29 features means better-quality synthetic data.

3.5.4 t-SNE

t-SNE [26] maps high-dimensional data to two dimensions for visualisation while trying to preserve the local neighbourhood structure. It places points that are close

in the original feature space close together in the 2D projection and separates clusters that are far apart. It is used to visually check whether synthetic samples land in the same regions as real fraud in the feature space, and to see whether fraud contains distinct sub-clusters that motivate the cluster-conditioning approach.

Summary

This chapter covered everything needed to understand the experimental setup: five classifiers of increasing complexity, six metrics suited to imbalanced classification, three oversampling baselines, and three generative model families. The next chapter applies all of this to the actual fraud datasets, running experiments and reporting results. Chapter ?? then builds on these foundations to introduce the proposed BD-WGAN architecture.

CHAPTER 4

Dataset, Preprocessing and Baseline Methods

Chapter 3 described the classifiers, metrics, and models used in this research. This chapter is where those tools are put to use. Two datasets are introduced, their properties are analysed, and then a sequence of experiments runs through five classifiers on raw imbalanced data, three traditional oversampling methods, and two GAN-based augmentation approaches. All results here form the comparison floor that the proposed BD-WGAN in Chapter ?? is evaluated against.

4.1 Dataset Description

4.1.1 ULB Credit Card Fraud Dataset

The main dataset used throughout this work is the ULB Credit Card Fraud Detection dataset [8], available on Kaggle. It contains transactions made with European credit cards over two consecutive days in September 2013.

- **Total transactions:** 284,807
- **Fraudulent:** 492 (0.1727%)
- **Legitimate:** 284,315 (99.8273%)
- **Features:** 30 (plus class label)
- **Recorded period:** approximately 48 hours

The 28 features labelled V1 through V28 are the result of a PCA transformation applied to the original transaction data. The raw features are not disclosed for confidentiality reasons. *Time* is the number of seconds from the first recorded transaction in the dataset. *Amount* is the transaction value in EUR. *Class* is the binary label (0 = legitimate, 1 = fraud).

The 577:1 imbalance ratio is the defining challenge. It is not a contrived scenario – it reflects how rare fraud genuinely is in practice. Working through the implications of this ratio is most of what this thesis is about.

4.1.2 IEEE-CIS Fraud Detection Dataset

The second dataset is the IEEE-CIS Fraud Detection dataset [15], released through a Kaggle competition run by IEEE Computational Intelligence Society and Vesta Corporation. It is used to test whether the proposed method works on a dataset with different properties from ULB.

The data arrives in two separate files. The transaction file has around 590,000 rows and 394 columns. The identity file adds 41 more columns for a subset of transactions – roughly 24% of them – and is joined by a shared transaction identifier. After merging, the combined dataset has approximately 590,000 rows and 430 columns before any feature selection.

Table 4.1: Class distribution in both datasets used in this thesis

Dataset	Fraud	Legitimate	Total	Ratio
ULB Credit Card	492	284,315	284,807	$\approx 577:1$
IEEE-CIS	$\approx 20,663$	$\approx 569,877$	$\approx 590,540$	$\approx 28:1$

The IEEE-CIS dataset is less severely imbalanced (28:1 versus 577:1) and has far more fraud examples (20,663 versus 492). The large number of features – many with substantial missing values – requires a dedicated feature engineering pipeline described in Section 4.3.

4.2 Exploratory Data Analysis

The EDA focuses on three questions relevant to the experiments: are there data quality issues that need fixing, which features best separate fraud from legitimate, and does fraud have internal structure that would motivate clustering?

4.2.1 Data Quality

The ULB dataset has no missing values across all 284,807 rows. All features are stored as float64. A duplicate check found 1,081 duplicate rows, every one of them in the legitimate class. These were removed during preprocessing. There are no duplicate fraud rows.

4.2.2 Discriminative Features

Since V1–V28 are PCA components of anonymised features, we cannot interpret them directly. The Kolmogorov-Smirnov test gives a way to rank them by how different

their distribution is between the fraud and legitimate classes. A high KS statistic means the two classes are far apart on that feature.

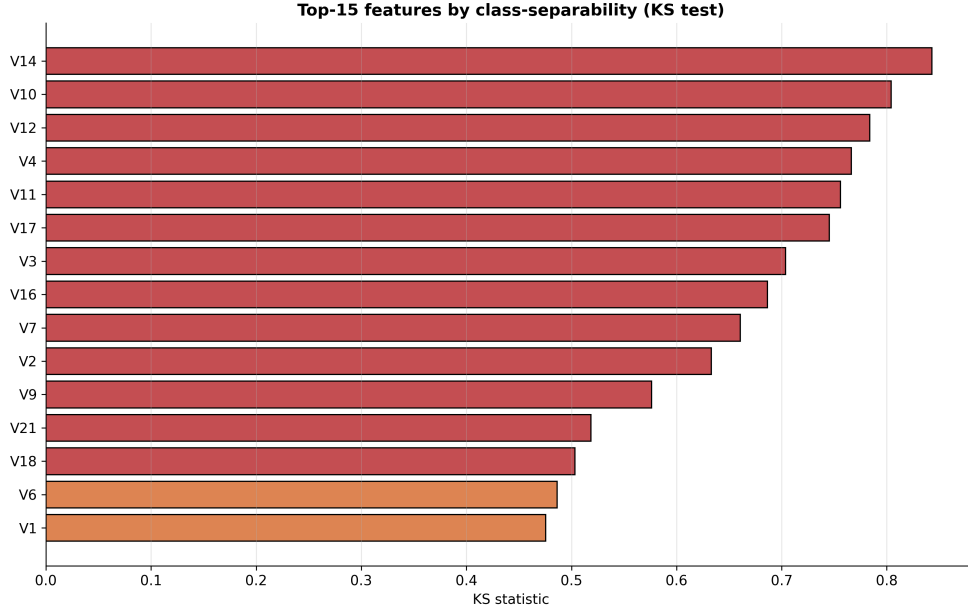


Figure 4.1: Features ranked by KS statistic between fraud and legitimate classes (higher = stronger separation)

V14, V4, V11, V12, and V10 come out at the top. These are the features where fraud and legitimate transactions look most different from each other, and they are expected to carry the most weight in any classifier. The Amount feature also shows a meaningful KS difference – fraud transactions cluster around smaller values on average.

4.2.3 Internal Structure of Fraud (t-SNE)

Whether fraud is one homogeneous group or several distinct patterns matters for the proposed generation approach. If fraud clusters into sub-groups, conditioning the generator on the cluster label should produce more realistic sub-group-specific samples.

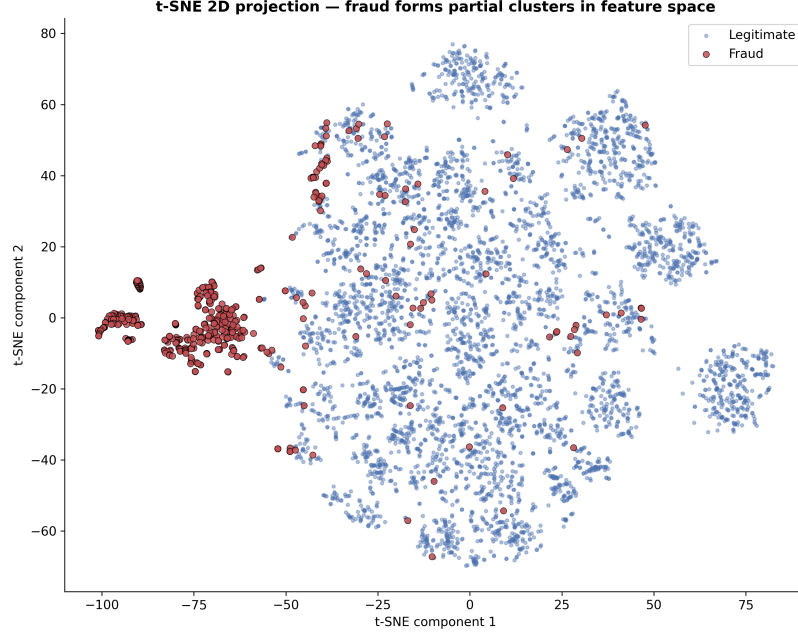


Figure 4.2: t-SNE projection: blue points are legitimate transactions, red are fraud. Fraud spreads across multiple separated regions rather than a single cluster.

The t-SNE plot makes it clear that fraud is not one tight cloud. The red points appear in several distinct regions of the 2D projection. This directly motivated the K-Means cluster-conditioning step in BD-WGAN.

4.3 Preprocessing Pipeline

4.3.1 ULB Preprocessing

The same four-step pipeline is applied to the ULB dataset in every experiment, without variation:

1. **Remove legitimate duplicates.** The 1,081 duplicate legitimate rows are dropped. All 492 fraud rows are kept, including any duplicates (none exist). The dataset is shuffled with random seed 42.
2. **Drop Time.** The Time feature records seconds from the first transaction. In its raw form it is not a useful classifier feature, and converting it to hour-of-day was done only for the EDA. It is dropped from the feature set.
3. **Scale Amount.** The Amount feature has a heavily skewed distribution with large outliers. RobustScaler is fitted on the training split only and applied to both splits. A separate StandardScaler is also fitted on the training split and applied to all 29 features (V1–V28 and the scaled Amount), producing the feature version used for logistic regression, KNN, and GAN training.

4. **Stratified split.** An 80/20 stratified split with random seed 42. The test set is never touched after this point.

Table 4.2: Training/test split after ULB preprocessing

Split	Total	Legitimate	Fraud
Training (80%)	226,997	226,602	394
Test (20%)	56,749	56,651	98

The test set has exactly 98 fraud cases. It is kept frozen through all 200+ experimental configurations.

4.3.2 IEEE-CIS Feature Engineering and Preprocessing

The IEEE-CIS dataset needs substantially more work before any model can use it. The pipeline runs ten steps:

1. **Merge.** Join transaction and identity files on TransactionID using a left join.
2. **Time features.** Extract hour-of-day, day-of-week, and week number from TransactionDT (a seconds offset). TransactionDT is then dropped.
3. **Log amount.** Compute $\log(1 + \text{TransactionAmt})$ to reduce the effect of extreme transaction values.
4. **Email domains.** Group the high-cardinality P_emaildomain and R_emaildomain columns into eight top domains plus a catch-all and a missing category. Add a binary flag for whether the purchaser and recipient domains match.
5. **Match flags.** Encode M1–M9 (True/False/NaN) as 1/0/−1 and add an M_sum column counting the number of True flags per row.
6. **Card and address frequency encoding.** Replace card1, card2, card3, card5, addr1, addr2 with their frequency of occurrence in the training set.
7. **D-feature imputation.** Median-impute all D1–D15 timedelta columns. Add binary missing-indicator flags for D1–D5.
8. **V-feature selection.** Drop any V feature with 70% or more missing values. Median-impute the rest. This retains roughly 100–120 V features out of 339.
9. **Identity encoding.** Impute numeric identity features (id_01 to id_38) with −999 as a sentinel. Frequency-encode categorical identity features and DeviceInfo/DeviceType.

10. **Remove near-constant features.** Drop any column with standard deviation below 0.01. Apply RobustScaler and then StandardScaler with the same train-only fitting rule as ULB. Stratified 80/20 split.

The result is a feature matrix with approximately 90–130 columns depending on how many V features survive the missing-value threshold in a given run.

4.4 Baseline Classifiers

Five classifiers are trained on the raw training data with no class weighting, no re-sampling, and no synthetic data. These are the “do nothing” numbers.

1. **Logistic Regression (LR):** L2 regularisation, max_iter=2000. StandardScaled features.
2. **K-Nearest Neighbours (KNN, $k = 5$):** StandardScaled features.
3. **Decision Tree (DT):** Default depth and splitting parameters.
4. **Random Forest (RF):** 200 trees, all other parameters default.
5. **XGBoost (XGB):** 200 estimators, histogram trees, no class weighting.

Table 4.3: Baseline results on the ULB test set (56,749 transactions, 98 fraud). No resampling applied. Bold marks the lowest FN.

Model	TP	FN	FP	Precision	Recall	F1
LR	63	35	10	0.8630	0.6429	0.7368
KNN	82	16	12	0.8723	0.8367	0.8542
DT	83	15	30	0.7345	0.8469	0.7867
RF	85	13	10	0.8947	0.8673	0.8808
XGB	80	18	8	0.9091	0.8163	0.8602

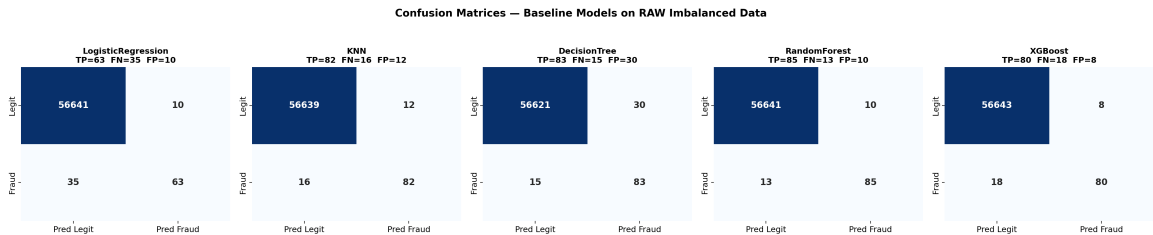


Figure 4.3: Confusion matrices for the five baseline classifiers on the ULB test set

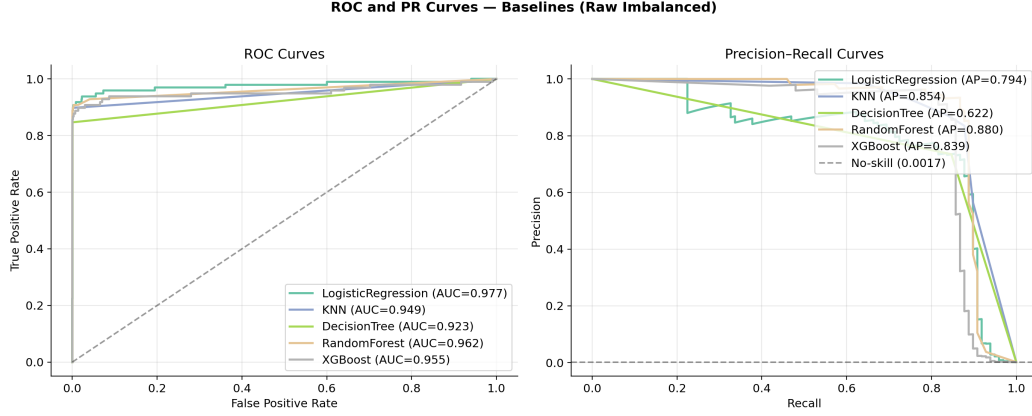


Figure 4.4: ROC and Precision-Recall curves for the five baseline classifiers

Random Forest achieves the lowest FN count (13 missed fraud out of 98) and the highest F1 (0.881). XGBoost gets the best precision (0.909) but misses 18 fraud cases. Logistic Regression drops sharply – 35 missed frauds, more than a third of the test fraud – because the linear decision surface cannot handle the non-linear fraud patterns in a 29-dimensional PCA space.

Even the best baseline (RF, FN = 13) leaves 13.3% of fraud undetected. In a real deployment this translates directly to financial losses that a bank has to cover. All subsequent methods try to push FN below this number.

4.5 Traditional Oversampling Methods

Three classical oversampling methods are applied to the training set only. The test set does not change. Each sampler is run at `sampling_strategy=1.0`, meaning fraud and legitimate counts are made equal in the training set before any classifier is trained. All five classifiers are retrained on the resampled data, giving 15 sampler-classifier combinations.

Random Over-Sampling (ROS) duplicates fraud samples at random. No new information is added.

SMOTE [6] creates new samples by interpolation between minority neighbours ($k = 5$). Diversity is added but stays within the convex hull of existing fraud.

ADASYN [14] uses the same interpolation as SMOTE but weights minority samples by how many majority neighbours surround them. Samples near the boundary get more synthetic neighbours.

Table 4.4: Full traditional resampling results on the ULB test set. Bold marks the lowest-FN result per sampler at Precision ≥ 0.70 . LR under ROS/ADASYN achieves FN=8 but at precision below 0.06 – over 1,000 false positives – which is not usable.

Sampler	Model	FN	Recall	Precision	F1	PR-AUC
ROS	LR	8	0.9184	0.0586	0.1102	0.7433
	KNN	12	0.8776	0.6825	0.7679	0.7940
	DT	19	0.8061	0.7117	0.7560	0.5741
	RF	14	0.8571	0.9231	0.8889	0.8775
	XGB	12	0.8776	0.8600	0.8687	0.9120
SMOTE	LR	10	0.8980	0.0546	0.1029	0.7453
	KNN	9	0.9082	0.4258	0.5798	0.6386
	DT	18	0.8163	0.3721	0.5112	0.3041
	RF	13	0.8673	0.8586	0.8629	0.8927
	XGB	12	0.8776	0.7544	0.8113	0.9071
ADASYN	LR	8	0.9184	0.0174	0.0341	0.7433
	KNN	9	0.9082	0.4258	0.5798	0.6335
	DT	18	0.8163	0.4278	0.5614	0.3495
	RF	14	0.8571	0.8750	0.8660	0.8787
	XGB	9	0.9082	0.7672	0.8318	0.9043

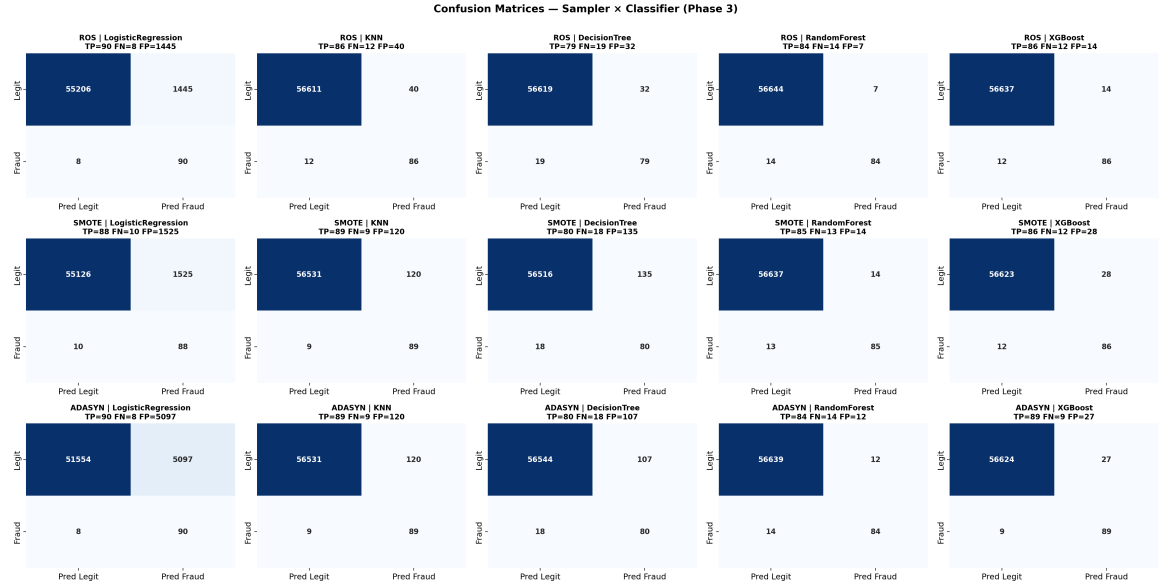


Figure 4.5: Confusion matrices for all 15 resampler × classifier combinations

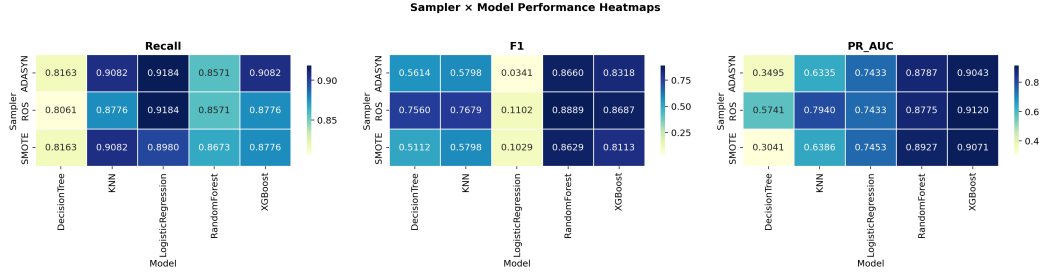


Figure 4.6: Performance heatmaps (Recall, F1, PR-AUC) across sampler-classifier combinations

The best result with a usable precision comes from ADASYN + XGBoost at $FN = 9$, $Recall = 0.908$, $Precision = 0.767$. This reduces missed fraud from 13 (best baseline) to 9 – a 31% reduction. ROS + XGBoost gets $FN = 12$, two better than the baseline. SMOTE + RF only matches the baseline ($FN = 13$) with slightly lower precision.

The LR results under ROS and ADASYN show something worth noting: both achieve $FN = 8$ with recall above 0.91, but with precision below 0.06, meaning for every fraud caught there are more than 15 false alarms. A bank’s fraud team cannot work at that ratio. A minimum usable precision of 0.70 is applied when selecting best configurations throughout this work.

All three methods share the same fundamental limitation. Their synthetic samples are linear combinations of existing training points. The multi-modal structure visible in the t-SNE plot of fraud (multiple separated clusters) cannot be reproduced by drawing straight lines between points from different clusters. A generative model that learns the actual fraud distribution can do better.

4.6 GAN-Based Oversampling Baselines

Two GAN architectures are evaluated as comparison baselines. Both are trained only on the 394 fraud training samples. The test set is never used in training. Each GAN is then used to generate synthetic fraud at five augmentation ratios (0.05, 0.10, 0.25, 0.50, 1.00 times the number of legitimate training samples), and all five classifiers are trained and evaluated at each ratio. This gives 50 configurations per GAN, 100 total.

4.6.1 Vanilla GAN

The vanilla GAN [11] uses binary cross-entropy as its loss. The generator maps a 32-dimensional noise vector through four fully-connected layers (with LeakyReLU activations and BatchNorm) to a 29-dimensional output. The discriminator uses three layers with Dropout.

Training: 2000 epochs, Adam with learning rate 2×10^{-4} , batch size 64. Training with only 394 samples is unstable – the discriminator tends to overfit quickly, which stalls the generator.

4.6.2 CTGAN

CTGAN [27] was configured with embedding dimension 64, generator and discriminator layer sizes (128, 128), 300 epochs, and batch size 50. The mode-specific normalisation helps it deal with the multi-modal distributions in the V features better than the vanilla GAN can.

4.6.3 Quality Check

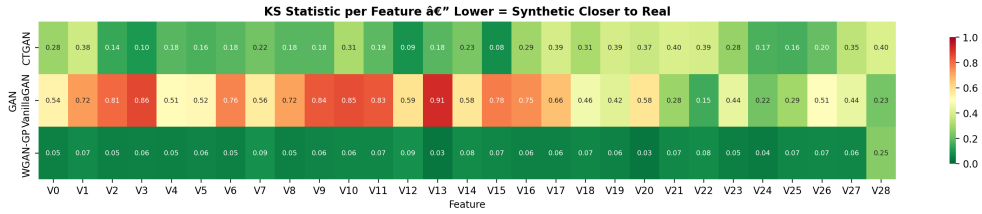


Figure 4.7: KS statistics per feature for each GAN (lower = generated distribution closer to real)

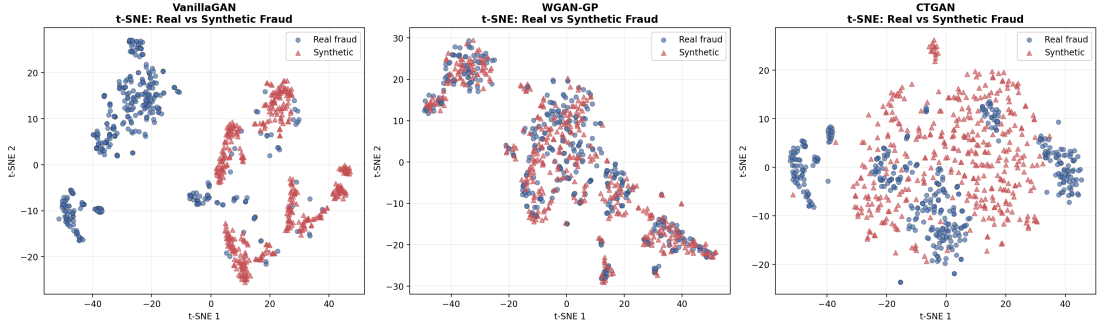


Figure 4.8: t-SNE projections comparing real fraud (blue) and synthetic fraud from each GAN (orange)

4.6.4 Results

Table 4.5: Best results per GAN across all 50 ratio $\times$ classifier configurations (Precision ≥ 0.70 , lowest FN).

GAN	Best Config	FN	TP	Recall	Precision	F1	PR-AUC
Vanilla GAN	ratio=0.25 + XGB	14	84	0.8571	0.9438	0.8984	0.8973
WGAN-GP	ratio=0.05 + KNN	11	87	0.8878	0.8131	0.8488	0.8082
CTGAN	ratio=0.05 + RF	11	87	0.8878	0.8447	0.8657	0.8882

WGAN-GP and CTGAN both reach FN = 11, two missed frauds fewer than the best baseline (RF, FN = 13) but two more than ADASYN + XGBoost (FN = 9). Vanilla GAN does worst at FN = 14 – worse than the best baseline – because its training on only 394 samples is too unstable to produce useful diversity.

GAN augmentation does improve over the classical baselines in F1 and precision, but it falls short of ADASYN on the raw FN count. The generated samples cover the average fraud distribution but not the hard boundary cases. Chapter ?? presents BD-WGAN, which addresses this directly.

Bibliography

- [1] R. Adhikari. Credit card fraud detection using conditional tabular gan and gradient boosting. *Journal of Financial Data Science*, 8(1):45–61, 2026.
- [2] Ahmad Baity Alfaizi and Suliman Mohamed Fati. An enhanced machine learning approach for credit card fraud detection using ensemble and deep learning. *IEEE Access*, 10:75921–75936, 2022.
- [3] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein GAN. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 214–223, 2017.
- [4] Siddhartha Bhattacharyya, Sanjeev Jha, Kurian Tharakunnel, and J. Christopher Westland. Data mining for credit card fraud: A comparative study. *Decision Support Systems*, 50(3):602–613, 2011.
- [5] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [6] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.
- [7] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- [8] Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson, and Gianluca Bontempi. Calibrating probability with undersampling for unbalanced classification. In *Proceedings of the IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 159–166, 2015.
- [9] European Central Bank. Report on card fraud. Technical report, European Central Bank, 2022. URL <https://www.ecb.europa.eu/pub/cardfraud/html/ecb.cardfraudreport202209.en.html>. Card-not-present fraud accounts for 79% of total fraud value in the European Union.
- [10] Ugo Fiore, Alfredo De Santis, Francesca Perla, Paolo Zanetti, and Francesco Palmieri. Using generative adversarial networks for improving classification effectiveness in credit card fraud detection. *Information Sciences*, 479:448–455, 2019.

- [11] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 2672–2680, 2014.
- [12] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of Wasserstein GANs. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5767–5777, 2017.
- [13] Haibo He and Edwardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009.
- [14] Haibo He, Yang Bai, Edwardo A. Garcia, and Shutao Li. ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In *Proceedings of the IEEE International Joint Conference on Neural Networks (IJCNN)*, pages 1322–1328, 2008.
- [15] IEEE Computational Intelligence Society and Vesta Corporation. IEEE-CIS Fraud Detection. Kaggle Competition Dataset, 2019. URL <https://www.kaggle.com/c/ieee-fraud-detection>.
- [16] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017.
- [17] Xu-Ying Liu, Jianxin Wu, and Zhi-Hua Zhou. Exploratory undersampling for class-imbalance learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 39(2):539–550, 2009.
- [18] Yang Liu, Wei Zhang, and Xin Chen. CTCN: Combining conditional tabular GAN with temporal convolutional networks for credit card fraud detection. *Expert Systems with Applications*, 225:120105, 2023.
- [19] Nelly Ngwenduna and Rendani Mbuva. Alleviating class imbalance in credit card fraud detection using generative adversarial networks. *PeerJ Computer Science*, 7:e604, 2021.
- [20] Smita Patil. Credit card fraud detection using CTGAN and machine learning. In *Proceedings of the International Conference on Machine Learning and Data Engineering (ICMLDE)*, 2022.
- [21] Apapan Pumsirirat and Liu Yan. Credit card fraud detection using deep learning based on auto-encoder and restricted Boltzmann machine. *International Journal of Advanced Computer Science and Applications*, 9(1):18–25, 2018.

- [22] Abhirup Roy, Jiyang Sun, Rachael Mahoney, Loris Alonzi, Stephen Adams, and Peter Beling. Deep learning detecting fraud in credit card transactions. *Proceedings of the IEEE Systems and Information Engineering Design Symposium (SIEDS)*, pages 129–134, 2018.
- [23] Yusuf Sahin and Ekrem Duman. Detecting credit card fraud by ANN and fuzzy logic. In *Proceedings of the International Symposium on Innovations in Intelligent Systems and Applications (INISTA)*, pages 315–319, 2011.
- [24] Abhinav Shrivastava, Abhinav Gupta, and Ross Girshick. Training region-based object detectors with online hard example mining. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 761–769, 2016.
- [25] The Nilson Report. Card fraud losses worldwide, 2024. URL <https://nilsonreport.com>. Issue 1268. HSN Consultants, Inc., Carpinteria, CA. Global card fraud losses reached \$33.83 billion in 2022 and are projected to exceed \$38.5 billion by 2027.
- [26] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008.
- [27] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. Modeling tabular data using conditional GAN. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.